

Aula 01 — Spring Framework

Introdução · Setup no Windows · Primeiro projeto

Mauricio Duailibi Neto

Entra21

Bem-vindos à última disciplina da trilha

Vocês já passaram por muita coisa:

- 1 Fundamentos de Programação
- 2 Orientação a Objetos
- 3 Java Web
- 4 **Spring Framework** ← estamos aqui

Promessa desta disciplina

Vamos criar aplicações web em Java de um jeito **mais prático** do que montar Servlet + Tomcat na mão — sem apagar o que vocês já aprenderam.

Mapa da aula de hoje

- 1 O que é Spring Boot?
- 2 Setup no Windows
- 3 Primeiro projeto (juntos)
- 4 Desafio (sozinhos)
- 5 Recapitulando

Lembra do Java Web?

Na disciplina anterior, para uma página responder no navegador, vocês tipicamente precisavam cuidar de várias peças:

- Instalar / configurar o **Tomcat**
- Criar **Servlets**
- Configurar o `web.xml` (em muitos projetos)
- Empacotar e publicar a aplicação

Funciona. Mas tem muita **montagem** antes do “olá, mundo”.

Analogia da cozinha

Java Web “na mão”

Você monta a cozinha:
fogão, pia, geladeira...
e **depois** cozinha.

Spring Boot

A cozinha já vem montada.
Você foca em **preparar o prato**
(a regra de negócio).

Frase para lembrar

Spring Boot **não substitui** Java.
Ele **organiza e acelera** o trabalho com Java.

Spring Conjunto grande de ferramentas para aplicações Java (web, dados, segurança...).

Spring Boot Forma **prática** de usar o Spring: configurações inteligentes + servidor embutido + projeto pronto para rodar.

Nesta disciplina, vamos falar principalmente de **Spring Boot**.

Quando alguém no mercado diz “trabalhei com Spring”, quase sempre envolve Spring Boot.

O que o Spring Boot faz por nós (versão simples)

- 1 Sobe um **servidor web embutido** (não precisamos instalar Tomcat separado)
- 2 Lê o `pom.xml` e baixa as **dependências**
- 3 Liga as peças da aplicação com menos arquivo de configuração manual
- 4 Deixa a gente escrever classes com anotações como `@RestController`

Meta de hoje

Criar um projeto, rodar, e ver uma mensagem no navegador.
Só isso — e isso já é uma vitória grande.

Comparação rápida (mental)

Ideia	Java Web	Spring Boot
Pedido HTTP	Servlet	Controller
Servidor	Tomcat externo	Embutido
Configuração	manual	automática
Dependências	Maven (pom.xml)	Maven (pom.xml)

Maven continua! Vocês já viram pom.xml. Isso ajuda bastante.

Checklist de instalação (Windows)

Vamos instalar / conferir **três coisas**:

- 1 **JDK 21** — a máquina virtual Java
- 2 **VS Code** — nosso editor
- 3 Extensões: **Java + Spring Boot**

Passo 1 — Instalar o JDK 21

- 1 Abra o navegador em:
`https://adoptium.net/`
- 2 Escolha:
 - Version: **21 (LTS)**
 - Operating System: **Windows**
 - Architecture: **x64** (maioria dos PCs)
- 3 Baixe o instalador `.msi` e execute
- 4 Aceite as opções padrão (marque a opção **Add to PATH** se aparecer)

Passo 1 — Conferir o Java

Abra o **Prompt de Comando** ou o **Terminal** do Windows:

```
java -version
```

Você deve ver algo parecido com:

```
openjdk version "21.0.x" ...
```

Se der “não é reconhecido como comando”

O Java não entrou no PATH. Reinstale marcando a opção de PATH ou peça ajuda ao professor antes de continuar.

Passo 2 — Instalar o VS Code

- 1 Acesse `https://code.visualstudio.com/`
- 2 Baixe para **Windows**
- 3 Instale com as opções padrão
- 4 (Opcional, mas útil) marque:
 - Abrir com Code no menu de contexto
 - Adicionar ao PATH

O VS Code é só o editor. As “superpoderes” Java vêm das **extensões**.

Passo 3 — Extensões no VS Code

No VS Code, clique no ícone de **Extensões** (ou Ctrl+Shift+X).

Instale estas duas:

- 1 **Extension Pack for Java** (Microsoft)
- 2 **Spring Boot Extension Pack** (VMware / Spring)

Espere terminar o download (pode demorar um pouco na primeira vez).

Dica

Reinicie o VS Code depois de instalar as extensões.

Preciso instalar Maven separado?

Não necessariamente.

Os projetos Spring Boot vêm com o **Maven Wrapper**:

- No Windows: `mvnw.cmd`
- Ele baixa o Maven automaticamente na primeira execução

Tradução

Vocês digitam um comando na pasta do projeto.
O próprio projeto cuida do Maven.

Checkpoint — estamos prontos?

Antes do primeiro projeto, confirme:

- java -version mostra 21 (ou 17+)

- VS Code abre normalmente

- Extension Pack for Java instalado

- Spring Boot Extension Pack instalado

Se algum item falhou: **chame o professor agora.**

Não siga adiante “no escuro”.

O que vamos construir juntos

Um projeto chamado **ola-spring** que:

- 1 Sobe um servidor na porta **8080**
- 2 Responde em `http://localhost:8080/`
- 3 Responde também em `http://localhost:8080/ola`

Sucesso de hoje

Ver uma frase escrita por vocês no navegador.
Isso prova que o Spring Boot está vivo na máquina.

Passo A — Abrir o Spring Initializr

O site que **gera o esqueleto** do projeto:

`https://start.spring.io`

Pensem nele como um “formulário de pedido”:
você escolhem o que querem, e ele monta a pasta inicial.

Passo B — Preencher o formulário

Configure assim (pode ir lendo em voz alta com a turma):

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** deixe a versão padrão do site
- **Group:** `com.entra21`
- **Artifact:** `ola-spring`
- **Name:** `ola-spring`
- **Package name:** `com.entra21.olaspring`
- **Packaging:** Jar
- **Java:** 21

Passo C — Adicionar a dependência certa

Em **Dependencies**, clique em **ADD DEPENDENCIES** e busque:

Spring Web

Adicione **só essa** por enquanto.

Por quê?

Sem Spring Web, o projeto nasce — mas **não** sobe endpoint HTTP pronto para o navegador do jeito que queremos hoje.

Passo D — Gerar e abrir no VS Code

- 1 Clique em **GENERATE**
- 2 Baixe o arquivo .zip
- 3 Extraia para uma pasta fácil, por exemplo:
C:\projetos\ola-spring
- 4 No VS Code: **File** → **Open Folder**
- 5 Selecione a pasta ola-spring (a que contém o pom.xml)

Armadilha comum

Abrir a pasta *pai* em vez da pasta do projeto.

O VS Code precisa “enxergar” o pom.xml na raiz aberta.

Passo E — Olhar a estrutura (sem medo)

```
ola-spring/  
  pom.xml                dependncias (Maven)  
  mvnw.cmd               Maven Wrapper (Windows)  
  src/main/java/com/entra21/olaspring/  
    OlaSpringApplication.java    ponto de partida (main)
```

Hoje só vamos **criar mais uma classe**: o Controller.

Passo F — Criar o OlaController

Na mesma pasta da Application, crie OlaController.java:

```
package com.entra21.olaspring;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class OlaController {

    @GetMapping("/")
    public String home() {
        return "Ola, Spring Boot! Bem-vindo(a).";
    }

    @GetMapping("/ola")
    public String ola() {
        return "Tudo funcionando!";
    }
}
```

O que essas anotações significam?

`@RestController` “Esta classe recebe pedidos HTTP e devolve resposta.”

`@GetMapping("/ola")` “Quando alguém acessar /ola com GET, rode este método.”

Ponte com Java Web

Isso lembra um Servlet mapeado para uma URL — só que com bem menos “cerimônia”.

Passo G — Rodar o projeto

No terminal do VS Code (Ctrl+` ou menu Terminal):

```
.\mvnw.cmd spring-boot:run
```

- A **primeira** vez demora (baixa bibliotecas)
- Espere aparecer algo como:
Started OlaSpringApplication

Não feche o terminal

Enquanto o servidor estiver rodando, o terminal fica “ocupado”.
Isso é normal. Para parar: Ctrl+C.

Passo H — Testar no navegador

Com o servidor rodando, abra:

- `http://localhost:8080/`
- `http://localhost:8080/ola`

O que é localhost:8080?

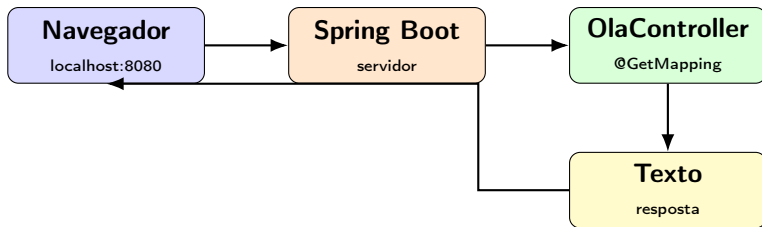
- **localhost** = este computador
- **8080** = porta onde o Spring está escutando

Se algo der errado...

- 1 Leia a mensagem vermelha do terminal (de baixo para cima)
- 2 Confira se o pacote das classes está igual (`com.entra21.olaspring`)
- 3 Confira se a dependência **Spring Web** foi adicionada
- 4 Veja se a porta 8080 já está em uso (feche outro Tomcat/Spring)
- 5 Peça ajuda — mostre a tela do erro

Erro não é fracasso. Erro é o computador pedindo ajuste.

Fluxo mental do que aconteceu



Vocês pedem → Spring recebe → Controller responde → navegador mostra.

Desafio 01 — Saudação personalizada

Criem um projeto (pode ser novo no Initializr) com:

Endpoint

```
GET /saudacao?nome=Maria
```

Resposta esperada:

```
Olá, Maria!  Seja bem-vindo(a) ao Spring.
```

Se abrir só /saudacao (sem nome):

```
Olá, visitante!  Seja bem-vindo(a) ao Spring.
```

Ferramenta nova: @RequestParam

Para ler ?nome=Maria da URL:

```
@GetMapping("/saudacao")
public String saudacao(
    @RequestParam(required = false) String nome) {

    if (nome == null || nome.isBlank()) {
        nome = "visitante";
    }
    return "Ola, " + nome + "! Seja bem-vindo(a) ao Spring.";
}
```

- required = false permite acessar sem o parâmetro
- O resto é Java que vocês **já sabem**

Critérios de aceite do desafio

Projeto sobe com `.\mvnw.cmd spring-boot:run`

`/saudacao?nome=Ana` mostra Ana

`/saudacao` mostra visitante

Vocês explicam, em uma frase, o que `@GetMapping` faz

Extra (opcional)

Criar `/soma?a=2&b=3` retornando 5.

Onde está o material

Tudo no repositório da disciplina:

- `slides/aula-01-introducao/` — estes slides
- `ola-spring/` — projeto feito juntos (consulta)
- `desafio-saudacao/` — enunciado do desafio

Use o projeto da aula como **gabarito vivo**, não como cópia cega: leiam, entendam, adaptem.

O que aprendemos hoje

- 1 Spring Boot acelera o que já vimos em Java Web
- 2 No Windows: JDK 21 + VS Code + extensões Java/Spring
- 3 O Initializr gera o esqueleto do projeto
- 4 @RestController + @GetMapping respondem no navegador
- 5 Rodamos com `.\mvnw.cmd spring-boot:run`

Para a próxima aula

- Tragam o JDK e o VS Code já funcionando
- Se puderem, terminem o desafio da saudação
- Na próxima: vamos aprofundar Controllers e organização do código

Obrigado — e até a próxima!